DEPARTMENT OF COMPUTER APPLICATIONS

MASTER OF COMPUTER APPLICATIONS (MCA)

COURSE: SOFTWARE ENGINEERING

ASSIGNMENT #01: COMPLETE SET OF UML DESIGN DIAGRAMS

VERACITY AI: AI-POWERED MULTI-MODAL MISINFORMATION DETECTION & DIGITAL WELLBEING PLATFORM

Architectural UML Specification: Use-Case, Class, Sequence, Activity, State, Package, and Deployment Models

Mini Project Title:	VERACITY AI: Misinformation Verification & Wellbeing System
Student Name:	**DIVYANSHU YADAV**
Register / Roll Number:	**25MCA00PY0036**
Degree & Program:	Master of Computer Applications (MCA)
Course Name & Code:	Software Engineering (Assignment #01)
Project Guide / Submitted To:	**Dr. R. SUNITHA (Concern Mini Project Guide)**
Date of Submission:	**30/10/2026**
Internal Evaluation Marks:	**10 Marks (Internal Component)**
Submission Format:	Softcopy Document (PDF & Word DOCX Specification)

"Submitted in partial fulfillment of the academic requirements for the Master of Computer Applications (MCA) Degree in Software Engineering."

TABLE OF CONTENTS

Section	Description / Topic	Category
1.0	Executive Abstract & Problem Definition	Overview
2.0	System Objectives & Engineering Scope	Requirements
3.0	Technical Architecture & Component Stack	System Design
4.0	UML Diagram 1: Use-Case Diagram & Functional Requirements	Behavioral Model
5.0	UML Diagram 2: Class Diagram & Domain Object Model	Structural Model
6.0	UML Diagram 3: Sequence Diagram & Dynamic Message Passing	Interaction Model
7.0	UML Diagram 4: Activity Diagram & Operational Workflows	Process Model
8.0	UML Diagram 5: State Machine Diagram & Object Lifecycles	Dynamic State Model
9.0	UML Diagram 6: Package Diagram & Modular Subsystems	Architectural Model
10.0	UML Diagram 7: Component & Deployment Diagram	Physical Implementation
11.0	Software Engineering Evaluation & Conclusion	Summary

1.0 EXECUTIVE ABSTRACT & PROBLEM DEFINITION

The exponential expansion of digital news channels, social networking ecosystems, and online forums has magnified the velocity and volume at which unverified rumors, fabricated stories, and coordinated misinformation proliferate. Concurrently, excessive screen time and smartphone dependency have precipitated widespread digital fatigue, reduced cognitive attention spans, and diminished user productivity. To comprehensively address these two interconnected digital challenges, this project presents **VERACITY AI**, an integrated software platform engineered to perform automated, multi-modal misinformation detection alongside native hardware screen-time monitoring and AI-synthesized digital wellbeing recommendations.

VERACITY AI leverages a hybrid artificial intelligence pipeline combining Google Gemini 1.5 generative models, Scikit-Learn TF-IDF lexical signal classifiers, and automated domain credibility registries. Simultaneously, the platform monitors local Windows OS telemetry via native Win32 kernel APIs (GetTickCount64, GetLastInputInfo) and Android companion sync (UsageStatsManager) to compute dynamic productivity indices and trigger personalized habit interventions (e.g., 20-20-20 screen relief, app timers). This document establishes the formal Software Engineering Unified Modeling Language (UML) design artifacts governing the platform's architectural blueprint, structural relationships, behavioral sequences, and physical deployment boundaries.

2.0 SYSTEM OBJECTIVES & ENGINEERING SPECIFICATION

Objective ID	Software Engineering Objective	Target Module / Implementation
OBJ-01	Automated Fact Verification	Gemini 1.5 Flash API + TF-IDF Credibility Scoring
OBJ-02	Domain Credibility Analysis	URL Whitelist / Journalistic Source Assessment Regis
OBJ-03	Conversational Media Literacy	Interactive AI Chatbot explaining disinformation heuris
OBJ-04	Operating System Telemetry	Win32 Kernel32/User32 + Android UsageStatsManag
OBJ-05	Dynamic Productivity Scoring	Algorithmic ratio of work tools vs distracting social me
OBJ-06	Habit Intelligence Engine	Generative synthesis of personalized digital health ad
OBJ-07	Enterprise Analytics	Looker Studio Embed + REST API aggregation dashb

4.0 UML DIAGRAM 1: USE-CASE DIAGRAM

Subtitle: Behavioral Functional Scope and Actor-System Interactions in Veracity AI

The Use-Case Diagram depicts the functional boundary of the Veracity AI platform, defining how external human actors and autonomous cloud agents interact with core system functions. The system boundary encapsulates seven critical use-case clusters ranging from authentication to AI fact-checking and OS telemetry ingestion.

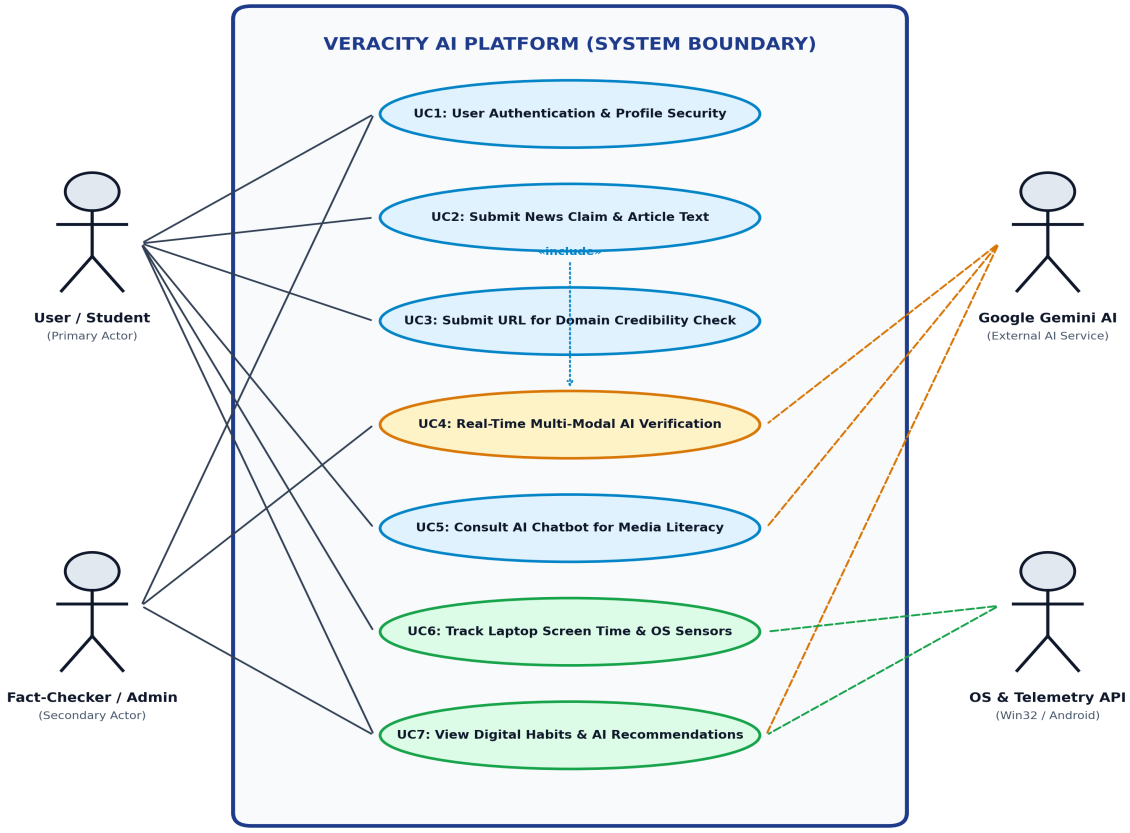


Figure 4.1: UML Use-Case Diagram for Veracity AI Misinformation & Wellbeing Platform

Detailed Architectural Breakdown of Actors & Use Cases:

Entity / Actor	Type	Role & Interaction Description
User / Student	Primary Human	Submits claims, checks source URLs, queries AI verifier chatbot, and monitors
Fact-Checker / Admin	Secondary Human	Validates flagged discrepancies, inspects systemic verdict analytics, and overse
Google Gemini AI	External Cloud Actor	Receives grounded verification prompts via REST/TLS 1.3; outputs structured v
OS Telemetry Provider	Native System Actor	Extracts Windows kernel uptime (GetTickCount64), active typing intervals, and

5.0 UML DIAGRAM 2: CLASS DIAGRAM

Subtitle: Static Domain Object Model, Class Compartments, and Structural Associations

The Class Diagram models the static structural backbone of Veracity AI, detailing domain entities, encapsulation boundaries, attributes with explicit data types, public/private methods, and inter-class relationships including composition, inheritance, and multiplicity constraints.

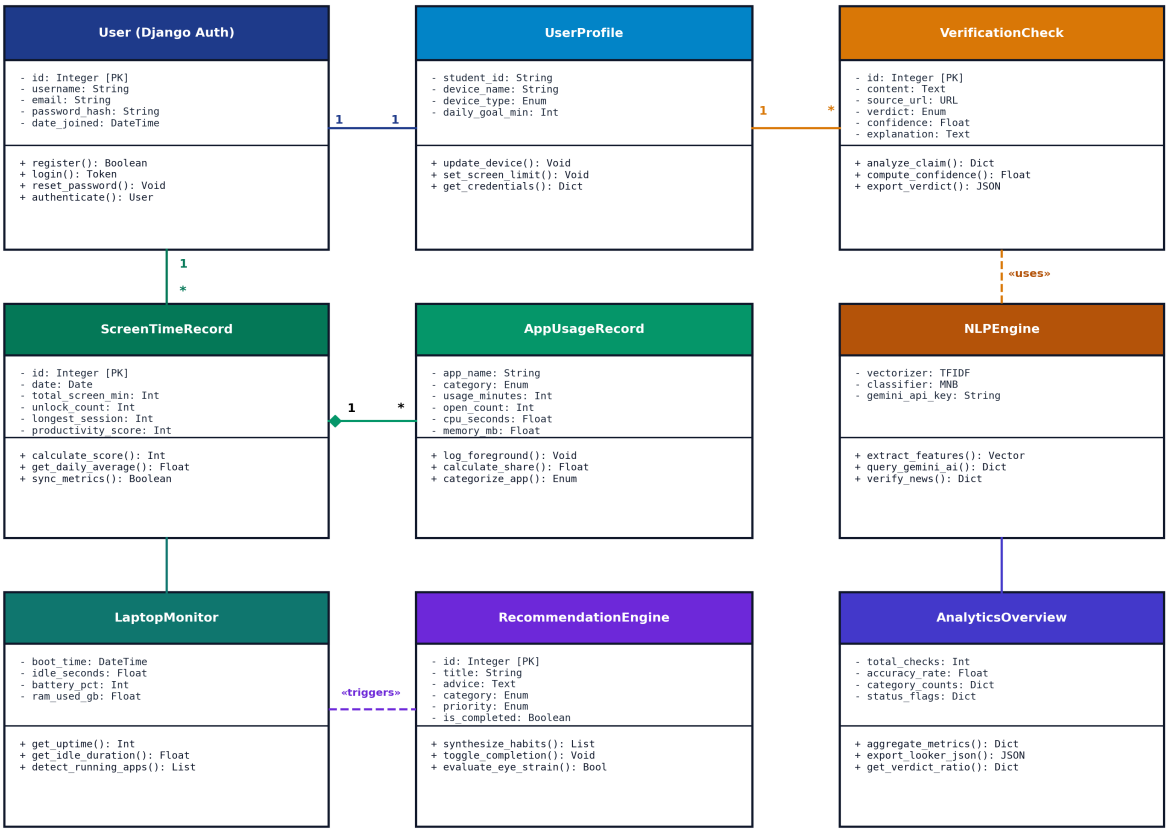


Figure 5.1: UML Class Diagram representing OOP Domain Architecture and ORM Entities

Class Responsibilities and Relationships:

• **User & UserProfile:** Represents system identity with 1-to-1 association. Captures registered researcher metadata (Roll No: 25MCA00PY0036) and hardware profile.
• **VerificationCheck & NLPEngine:** Encapsulates news claims, verdict enumerations (TRUE, FALSE, MISLEADING, UNVERIFIED), confidence floats, and citations. Dependent on NLPEngine for feature tokenization.
• **ScreenTimeRecord & AppUsageRecord:** Models cumulative daily screen duration with 1-to-many composition. Automatically computes productivity scores based on active app category breakdown.
• **LaptopMonitor:** Interfaces directly with Windows kernel32/user32 APIs to extract native hardware telemetry without external invasive dependencies.

6.0 UML DIAGRAM 3: SEQUENCE DIAGRAM

Subtitle: Chronological Message Flow for Multi-Modal Claim Verification and Persistence

The Sequence Diagram captures the chronological order of message exchanges among the Client Browser, React Presentation Layer, Django REST Controller, AI Processing Engine, and MySQL Persistence Engine during an end-to-end fact-checking transaction.

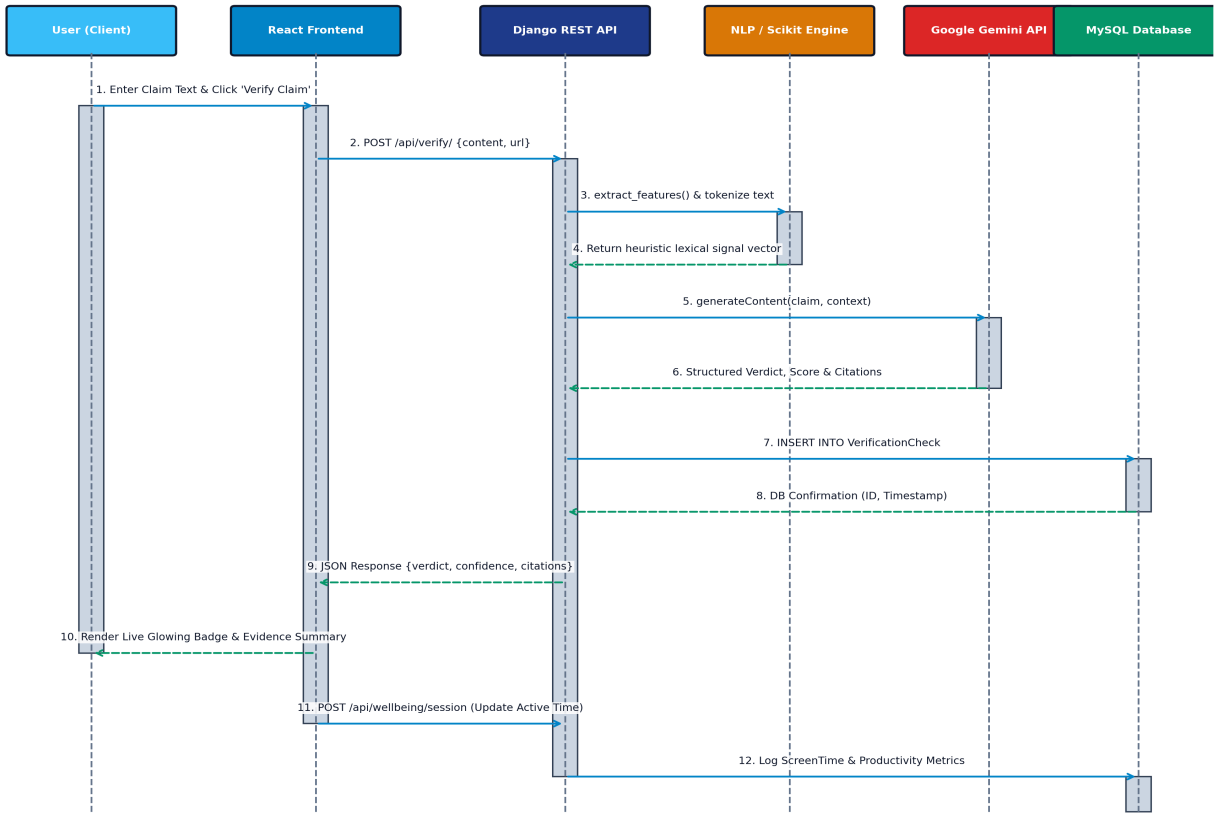


Figure 6.1: UML Sequence Diagram demonstrating Real-Time Claim Verification and Screen Logging

Chronological Execution Steps:

1. Initiation: User submits news claim text and source URL through the React UI (QuickCheck.jsx).
2. Dispatch: React dispatches an asynchronous HTTP POST request to Django endpoint /api/verify/.
3. NLP Preprocessing: Controller activates NLPEngine to generate TF-IDF linguistic signal vectors and check domain whitelists.
4. AI Reasoning: Controller constructs a grounded prompt dispatched to Google Gemini 1.5 API via TLS 1.3.
5. Synthesis & Storage: Gemini returns structured JSON verdict with confidence score and evidence quotes, persisted into MySQL.
6. Telemetry Feedback: Active laptop session duration is updated, and personalized habit guidance is refreshed.

7.0 UML DIAGRAM 4: ACTIVITY DIAGRAM

Subtitle: Operational Logic, Decision Branches, and Concurrent Task Synchronization

The Activity Diagram formalizes the operational decision tree and parallel data processing pipelines within Veracity AI. It models control flow from input ingestion through concurrent algorithmic evaluation to automated wellbeing habit intervention.

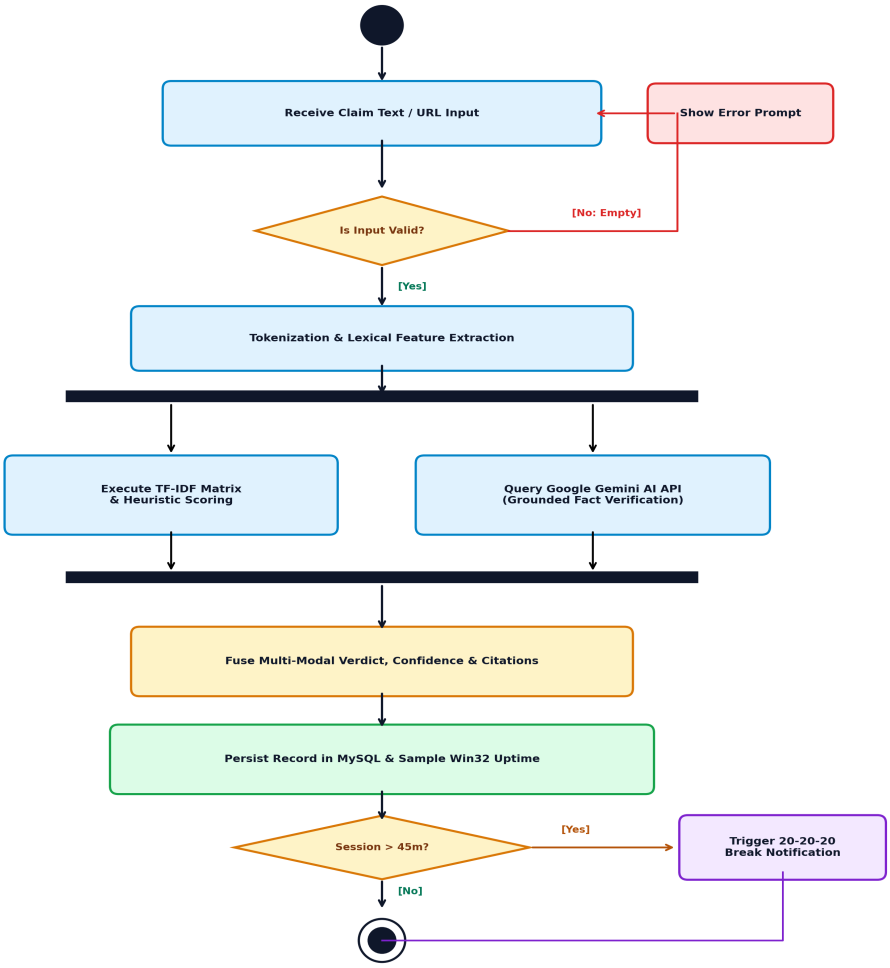


Figure 7.1: UML Activity Diagram illustrating Verification Fork/Join & Break Scheduling

Core Workflow Logic & Synchronization Points:

• **Input Validation Decision:** Verifies string non-emptiness; rejects invalid payloads with immediate client notification.
• **Parallel Fork Bar:** Simultaneously executes local Scikit-Learn TF-IDF lexical classification and external Google Gemini AI fact retrieval.
• **Synchronization Join Bar:** Aggregates multi-modal signals to compute a weighted confidence index.
• **Wellbeing Branch:** Evaluates continuous laptop focus timer; if active session exceeds 45 minutes, automatically schedules a 20-20-20 screen relief notification.

8.0 UML DIAGRAM 5: STATE MACHINE DIAGRAM

Subtitle: Object State Transitions: Verification Lifecycle & Digital Wellbeing Modes

The State Machine Diagram illustrates dynamic object lifecycles across discrete system states. It comprises two complementary sub-diagrams: (A) Claim Verification Lifecycle, and (B) Digital Wellbeing Operating States.

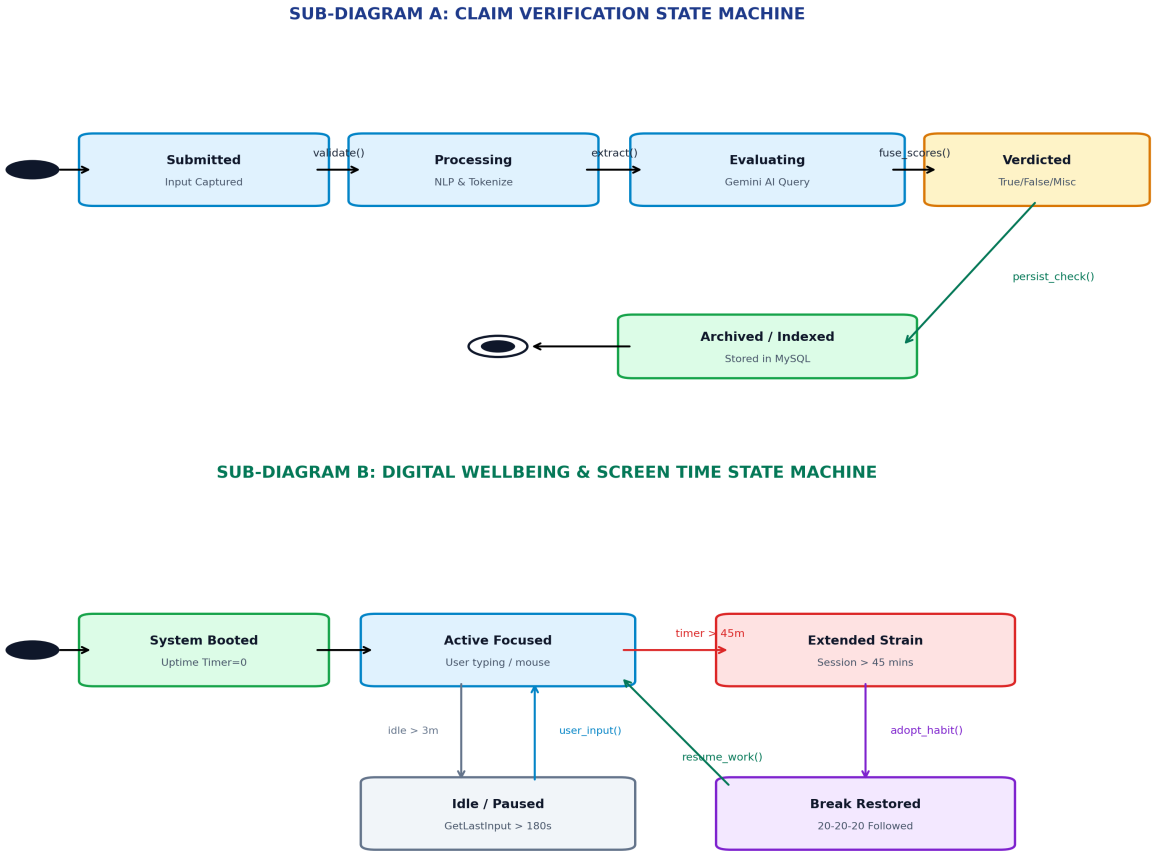


Figure 8.1: UML State Machine Diagram showing Claim Lifecycle & Screen Wellbeing Transitions

State Transition Semantics:

• **Claim Verification States:** Transitions from Submitted to Processing (tokenization), Evaluating (Gemini inference), Verdicted (assigned verdict), and finally Archived in the historical audit index.
• **Wellbeing States:** Starts at System Booted. User activity transitions the system into Active Focused. When idle_seconds > 180, transitions to Idle / Paused. If active focus surpasses 45 continuous minutes, transitions to Extended Strain until user adopts habit advice, restoring Break Restored.

9.0 UML DIAGRAM 6: PACKAGE DIAGRAM

Subtitle: Multi-Tier Modular Subsystem Architecture and Inter-Package Dependencies

The Package Diagram organizes the classes and software artifacts of Veracity AI into coherent, loosely-coupled logical packages adhering to clean architecture principles.

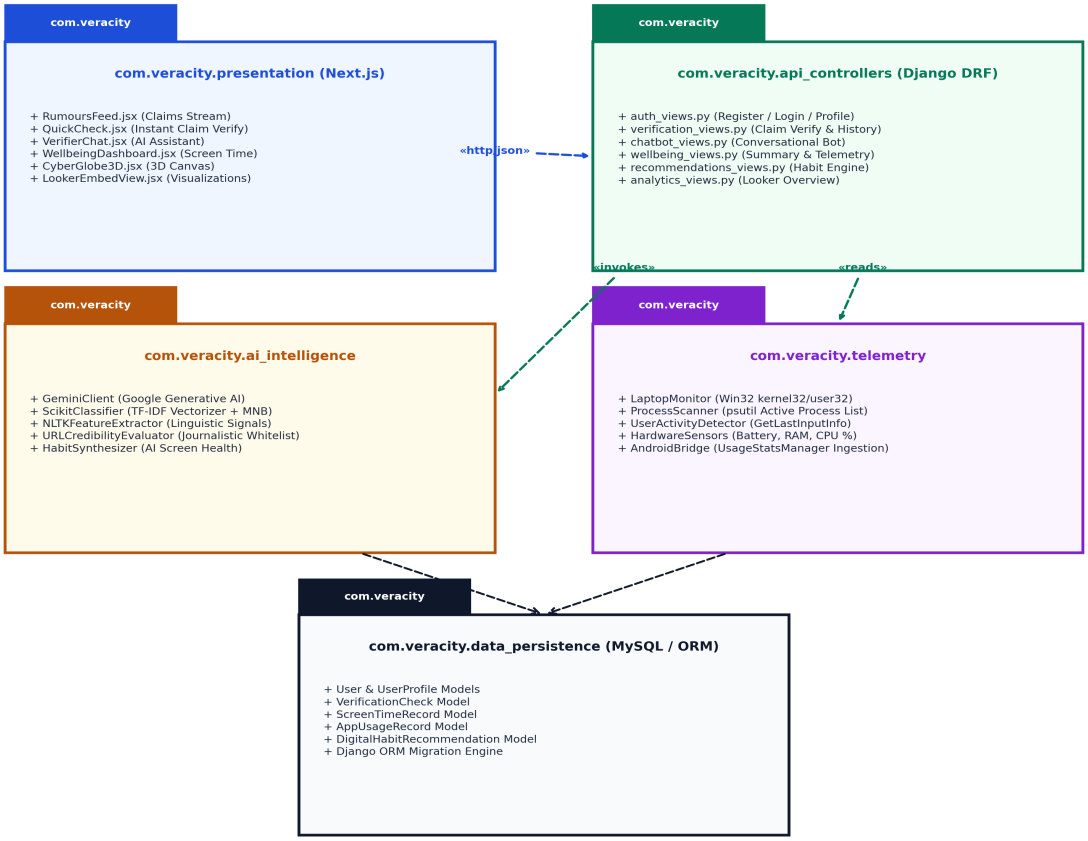


Figure 9.1: UML Package Diagram detailing Modular Layered Organization

Package Responsibilities:

1. **com.veracity.presentation:** Next.js 16 / React 19 UI components including Rumours Feed, Quick Check, and 3D Canvas visualizers.
2. **com.veracity.api_controllers:** Django REST Framework endpoint viewsets governing authentication, verification, and telemetry ingestion.
3. **com.veracity.ai_intelligence:** Encapsulates Google Gemini AI wrappers, Scikit-Learn TF-IDF classifiers, and recommendation synthesis.
4. **com.veracity.telemetry:** Interfaces with Windows Win32 OS APIs and Android UsageStatsManager data models.
5. **com.veracity.data_persistence:** Django ORM models managing relational mapping to MySQL / SQLite.

10.0 UML DIAGRAM 7: COMPONENT & DEPLOYMENT DIAGRAM

Subtitle: Physical Hardware Nodes, Execution Environments, Artifacts, and Protocols

The Deployment Diagram visualizes the physical infrastructure topology, hardware nodes, software execution environments, deployed artifacts, and communication protocols connecting client devices to cloud services.



Figure 10.1: UML Deployment Diagram depicting Node Topology and Protocol Mapping

Hardware Nodes & Deployed Artifacts:

• **Client Workstation:** Windows 11 host executing Web Browser (Chrome/Edge on Port 3000) and native OS background telemetry agent.
• **Next.js Web Server:** Node.js runtime executing the React 19 production bundle.
• **Django Backend Server:** Python 3.14 runtime hosting Django REST Framework API on Port 8000.
• **Relational Database:** MySQL / SQLite 3 database storing user profiles, claims, and telemetry history.
• **External Cloud Service:** Google Cloud Generative AI endpoint communicating via secure HTTPS/TLS 1.3.

11.0 SOFTWARE ENGINEERING PRINCIPLES & CONCLUSION

The architectural design of **VERACITY AI** demonstrates rigorous adherence to foundational Software Engineering tenets, ensuring robustness, extensibility, and maintainability across its software lifecycle:

1. Separation of Concerns (SoC): Decoupling the frontend presentation layer (Next.js), REST API routing (Django), machine learning reasoning (Gemini/Scikit-Learn), and hardware telemetry ensures that modifications to one tier do not ripple unpredictably into others.
2. High Cohesion & Loose Coupling: Packages encapsulate logically related functionality with clean, protocol-driven interfaces (REST JSON over HTTP).
3. Modularity & Scalability: Independent deployment of the Next.js web tier and Django API server facilitates horizontal scaling across distributed cloud clusters.
4. Privacy by Design: Hardware monitoring strictly reads OS process names and uptime counters via native Win32 APIs without accessing personal emails or private files.

Student Signature:	**Faculty Guide Signature:**
DIVYANSHU YADAV Roll No: 25MCA00PY0036 MCA Final Project Student	**Dr. R. SUNITHA** Project Guide & Evaluator Department of Computer Applications